PART 2 OF 2

BrainBarter

Complete Code Explained — Part 2

Every file, every function, every line — explained so clearly you can answer any question an interviewer throws at you.

Frontend Pages | Layout Components | Common Components | Server Routes | Database SQL | 30 Interview Q&As

Author: Abhay Kumar Dwivedi | Stack: React + Vite + Node.js + Supabase + pgvector + Gemini AI

This document is Part 2. Read Part 1 (brainbarter_explained.pdf) first for server controllers, utils, stores, and App-level files.

Table of Contents

Leaderboard.jsx

Database

COMPLETE_SETUP.sql — full schema, RLS, RPCs

Interview Prep

30 Interview Questions & Answers

client/src/main.jsx App Entry Point React 18

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

createRoot: React 18's new root API. Instead of the old `ReactDOM.render()`, we use `createRoot(domNode).render(jsx)`. This enables concurrent features like Suspense, transitions, and automatic batching of state updates.

document.getElementById('root'): Grabs the `<div id="root">` from `index.html`. React mounts the entire app inside that div — everything you see on screen is React-rendered inside it.

StrictMode: A development-only wrapper. It deliberately renders every component TWICE to expose side-effects in `useEffect`, and warns about deprecated APIs. It has ZERO effect in production — no double renders in prod. This is why you might see console logs appearing twice during development.

index.css: Imported here so it applies globally. It contains Tailwind's base/components/utilities layers and custom CSS classes like `.card`, `.btn-primary`, `.skeleton`, etc.

server/routes/auth.js Auth Routes Express Router

```
const express = require('express')
const { sendVerificationCode, verifyCode } = require('../controllers/authController')
const router = express.Router()

router.post('/send-verification', sendVerificationCode)
router.post('/verify-code', verifyCode)
```

```
module.exports = router
```

express.Router(): Creates a mini-app that handles its own routes. This router is mounted in `server/index.js` at `app.use('/api/auth', authRoutes)`, so the full URL becomes `/api/auth/send-verification`.

No verifyToken middleware here — and that's intentional! During registration/login, the user doesn't have a JWT yet. These endpoints are public. Once the user registers and logs in, they get a Supabase JWT, which ALL other routes require.

Controller pattern: The router file is thin — it only defines which function handles which HTTP method + path. All the logic lives in the controller.

`server/routes/ai.js` AI Routes

Express Router

```
const express = require('express')
const router  = express.Router()
const { verifyToken }    = require('../middleware/verifyToken')
const { assist, examMode, ingestContent } = require('../controllers/aiController')

router.post('/assist',    verifyToken, assist)
router.post('/exam',      verifyToken, examMode)
router.post('/ingest',    verifyToken, ingestContent)

module.exports = router
```

verifyToken as middleware: Notice how `verifyToken` appears BETWEEN the route path and the handler function: `router.post('/assist', verifyToken, assist)`. Express calls middleware functions left to right. So: request arrives → verifyToken runs → if valid, calls `next()` → assist runs. If invalid JWT, verifyToken sends 401 and assist never runs.

/assist: Used by ContentPage.jsx for AI features on specific content (summarize, doubt, generate notes).

/exam: Used by ExamMode.jsx for subject-level AI (expected questions, mock test, study plan).

/ingest: Called fire-and-forget after upload. Extracts PDF text, chunks it, embeds with Gemini, stores vectors in Supabase. Enables RAG for the AI assistant.

`server/routes/payment.js` Payment Routes

Express Router

```
const express = require('express')
const router  = express.Router()
const { verifyToken } = require('../middleware/verifyToken')
const { createOrder, verifyPayment } = require('../controllers/paymentController')

router.post('/create-order', verifyToken, createOrder)
router.post('/verify',       verifyToken, verifyPayment)

module.exports = router
```

Both endpoints are protected — you must be logged in to buy tokens. This prevents anonymous payment orders and ensures we know which user's balance to credit.

Two-step payment flow: `/create-order` talks to Razorpay API to create an order (returns an order_id). Wallet.jsx opens the Razorpay checkout UI with that order_id. After payment, Razorpay calls our `/verify` endpoint with the payment signature. We verify it server-side to prevent fake payments.

Why two endpoints? Security. Token amounts are locked server-side when the order is created — the client can't tamper with the amount. The signature verification prevents replaying old payments or crafting fake ones.

`client/src/components/layout/Navbar.jsx` Navbar

Layout Component

```
const navLinks = [
  { to: '/',            label: 'Home',      icon: RiHome4Line },
  { to: '/browse',      label: 'Browse',    icon: RiCompassLine },
  { to: '/exam-mode',   label: 'Exam Mode', icon: RiFlashlightLine },
```

```
  { to: '/leaderboard', label: 'Leaders',   icon: RiTrophyLine },
]
```

Data-driven nav: Links are defined as an array of objects and rendered with `.map()`. Adding a new nav item only requires adding one object to this array — no JSX changes needed. This is cleaner than hardcoding four separate `<Link>` elements.

```
const isActive = (to) => location.pathname === to
```

useLocation(): From React Router. Returns the current URL's pathname. `isActive` compares it to each link's `to` path. When it matches, the link gets the purple active style; otherwise the gray hover style. This is how the nav highlights the current page.

```
const handleLogout = async () => {
  await supabase.auth.signOut()
  logout()
  ...
  navigate('/')
}
```

Two-part logout: First `supabase.auth.signOut()` — this invalidates the session in Supabase and removes the token from localStorage. Then `logout()` — this clears the Zustand store (user=null, profile=null). Both are needed: without supabase.signOut(), the localStorage token persists and the user gets auto-logged back in on next refresh.

```
<div className="fixed inset-0 z-10" onClick={() => setUserMenuOpen(false)} />
```

Click-outside to close dropdown: When the user menu is open, a transparent full-screen overlay is placed behind the dropdown. Clicking anywhere outside the menu clicks this invisible div, which calls `setUserMenuOpen(false)`. Classic UX pattern — no complex event propagation needed.

```
{profile?.role === 'admin' && (
  <Link to="/admin">Admin Panel</Link>
)}
```

Role-based UI: The Admin Panel link only renders if `profile.role === 'admin'`. This is checked from the Zustand store (which loaded from Supabase after login). The route `/admin` is also protected server-side by ProtectedRoute with `adminOnly`, so even if someone manually navigates there, they can't access it without the role.

```
<nav className="sticky top-0 z-40 bg-white/80 dark:bg-[#0f0a1e]/80 backdrop-blur-md ...">
```

Glassmorphism nav: `sticky top-0` keeps it pinned as you scroll. `bg-white/80` means white with 80% opacity. `backdrop-blur-md` blurs whatever is behind it. Together these create the frosted-glass look where you can faintly see content scrolling under the nav.

`client/src/components/layout/Footer.jsx` **Footer** Layout Component

```
export default function Footer() {
  return (
    <footer className="border-t ... mt-auto">
      <div className="container-app py-8">
        Logo · Tagline · Social links
      </div>
    </footer>
  )
}
```

Simple presentational component. `mt-auto` pushes it to the bottom because App.jsx wraps everything in a `flex flex-col min-h-screen` div with `<main className="flex-1">`. The flex-1 main takes up all available height, and footer naturally sits at the bottom.

client/src/components/common/Button.jsx

Button Component

Reusable UI

```
const variants = {
  primary:   'btn-primary',
  secondary: 'btn-secondary',
  ghost:     'btn-ghost',
  danger:    'btn-danger',
  outline:   'btn border-2 bg-transparent',
}
const sizes = {
  sm: 'btn-sm', md: '', lg: 'btn-lg', icon: 'btn-icon',
}

export default function Button({ children, variant = 'primary', size = 'md',
  className = '', loading = false, ...props }) {
  return (
    <button
      className={`btn ${variants[variant]} ${sizes[size]} ${className}`}
      disabled={loading || props.disabled}
      {...props}
    >
      {loading && <span className="w-4 h-4 border-2 border-current border-t-transparent rounde
      {children}
    </button>
  )
}
```

variant lookup: Instead of if/else chains, we use a plain object as a lookup table. `variants[variant]` returns the CSS class string. Default is `'primary'`.

loading spinner: When `loading={true}`, a small spinning circle appears inline before children. The button is also disabled via `disabled={loading || props.disabled}` to prevent double-submits.

...props spread: Any extra HTML attribute (onClick, type, disabled, etc.) is forwarded to the button element via rest props. This means `<Button type="submit">` works without extra code.

className prop: Allows callers to add extra Tailwind classes on top of the base button styles: `<Button className="w-full">`.

client/src/components/common/Input.jsx

Input Component

Reusable UI

```
export default function Input({ label, error, className = '', icon: Icon, ...props }) {
  return (
    <div className="w-full">
      {label && <label className="input-label">{label}</label>}
      <div className="relative">
        {Icon && (
          <span className="absolute left-3 top-1/2 -translate-y-1/2 text-gray-400">
            <Icon size={16} />
          </span>
        )}
        <input
          className={`input ${Icon ? 'pl-9' : ''} ${error ? 'border-red-400 focus:ring-red-200
          {...props}
        />
      </div>
      {error && <p className="input-error">{error}</p>}
    </div>
  )
}
```

icon: Icon — Destructuring with rename. The prop is called `icon` (lowercase) but renamed to `Icon` (uppercase) because React requires component names to start with uppercase. Usage: `<Input icon={RiMailLine} />`.

pl-9: Left padding of 36px to make room for the icon so they don't overlap.

Absolute positioning: The icon sits absolutely inside the relative container, centered vertically with `top-1/2 -translate-y-1/2`.

error state: Red border + red ring on focus. Error message renders below the input only when `error` prop is truthy.

client/src/components/common/Avatar.jsx

Avatar Component

Reusable UI

```
const initials = name.split(' ').map(w => w[0]).join('').slice(0, 2).toUpperCase()

if (src) {
  return <img src={src} alt={name} className={`${sizes[size]} rounded-full object-cover ...`}
}
```

```
  return (
    <div className={`${sizes[size]} rounded-full bg-gradient-brand flex items-center justify-cen
      {initials || '?'}
    </div>
  )
```

Initials generation: Split name by space → take first character of each word → join → max 2 characters → uppercase. "Abhay Kumar" → ["A","K"] → "AK".

Fallback: If `src` is provided, show the image. Otherwise show initials on a purple gradient background. If name is empty, show "?" as fallback.

object-cover: For images, fills the circular space without distorting the aspect ratio (crops if needed).

`client/src/components/common/Badge.jsx` Badge Component

Reusable UI

```
const variants = {
  purple: 'badge-purple', green: 'badge-green', amber: 'badge-amber',
  red: 'badge-red', gray: 'badge-gray',
}
export default function Badge({ children, variant = 'purple', className = '' }) {
  return <span className={`${variants[variant]} ${className}`}>{children}</span>
}
```

Tiny component — only 10 lines. Maps a variant name to a CSS class. The actual colors (background, text) are defined in `index.css` as `.badge-purple { ... }`. Usage: `<Badge variant="green">beginner</Badge>`.

`client/src/components/common/TokenChip.jsx` TokenChip Component

Reusable UI

```
import { RiCopperCoinLine } from 'react-icons/ri'

export default function TokenChip({ amount, className = '' }) {
  return (
    <span className={`token-chip ${className}`}>
      <RiCopperCoinLine size={14} />
      {amount}
    </span>
  )
}
```

The coin icon + number combination appears everywhere — content cards, Navbar balance, content unlock button. Centralizing it as a component means if we change the icon or styling, it updates everywhere instantly.

`client/src/components/common/Modal.jsx` Modal Component

Reusable UI

```
useEffect(() => {
  if (open) document.body.style.overflow = 'hidden'
  else document.body.style.overflow = ''
  const onKey = (e) => { if (e.key === 'Escape' && open) onClose?.() }
  window.addEventListener('keydown', onKey)
  return () => {
    document.body.style.overflow = ''
    window.removeEventListener('keydown', onKey)
  }
}, [open, onClose])
```

Body scroll lock: When a modal is open, we don't want the background page to scroll. `document.body.style.overflow = 'hidden'` prevents that. The cleanup function (returned from useEffect) restores it when the modal closes or the component unmounts.

Escape key: The keyboard event listener enables pressing Escape to close the modal. `onClose?.()` uses optional chaining — if `onClose` is not provided, it silently does nothing instead of throwing.

Cleanup: The return function removes the event listener. Without this, every time the modal opens and closes, a new listener is added but old ones stack up (memory leak).

```
if (!open) return null
```

Early return pattern. If modal is not open, nothing renders at all — no DOM nodes, no z-index issues. React unmounts and remounts children on open/close, so state inside modals resets automatically.

```
<div className="absolute inset-0 bg-black/40 backdrop-blur-sm animate-fade-in" onClick={onClos
```

Backdrop click to close: The semi-transparent dark overlay behind the modal is a div that covers the full screen. Clicking it triggers `onClose`. This is separate from the white modal card so clicking the modal itself doesn't close it.

`client/src/components/common/Skeleton.jsx`

Skeleton & CardSkeleton

Loading UI

```
export default function Skeleton({ className = '', rows = 1 }) {
  if (rows > 1) {
    return (
      <div className="space-y-3">
        {Array.from({ length: rows }).map((_, i) => (
          <div key={i} className={`skeleton h-4 ${i === rows - 1 ? 'w-3/4' : 'w-full'} ${class
        ))}
      </div>
    )
  }
  return <div className={`skeleton ${className}`} />
}

export function CardSkeleton() {
  return (
    <div className="card space-y-4">
      <Skeleton className="h-40 w-full rounded-xl" />
      <Skeleton className="h-4 w-3/4" />
```

```
      <Skeleton className="h-3 w-1/2" />
      <div className="flex gap-2">
        <Skeleton className="h-6 w-16 rounded-full" />
        <Skeleton className="h-6 w-16 rounded-full" />
      </div>
    </div>
  )
}
```

Skeleton loading: Instead of showing a spinner while content loads, we show gray animated placeholder shapes that match the layout of the real content. Users see something immediately and don't experience layout shift when content appears.

Array.from({ length: rows }): Creates an array of `rows` empty slots to map over. Used to render multiple skeleton lines without hardcoding them.

Last line shorter (w-3/4): Real text lines are rarely all the same length — the last line is often shorter. This mimics that natural pattern, making skeletons look more realistic.

CardSkeleton: A composite skeleton that looks like a content card — image placeholder, title, subtitle, two badge shapes. Used in Browse.jsx while content loads.

.skeleton CSS class: Defined in index.css with a shimmer animation (gradient moving left to right) to give the "loading" feel.

`client/src/components/common/Loader.jsx`

Loader & PageLoader

Loading UI

```
export default function Loader({ size = 'md', className = '' }) {
  const s = { sm: 'w-4 h-4 border-2', md: 'w-8 h-8 border-2', lg: 'w-12 h-12 border-3' }
  return <div className={`${s[size]} border-brand-200 border-t-brand-600 rounded-full animate-
}

export function PageLoader() {
  return (
    <div className="min-h-screen flex items-center justify-center ...">
      <div className="flex flex-col items-center gap-4">
        <div className="w-12 h-12 border-3 border-brand-200 border-t-brand-600 rounded-full an
        <p className="text-sm text-gray-500 ... animate-pulse-slow">Loading...</p>
      </div>
    </div>
```

```
  )
}
```

CSS spinner technique: A div with `rounded-full` (circle), `border` (ring), but `border-t-brand-600` (only top border colored). With `animate-spin`, the colored top border rotates creating the spinning loader effect.

PageLoader: Full-screen centered loader. Used by ProtectedRoute while waiting for auth state to confirm, and by AdminPanel during data loading.

Named export + default export: The file exports both a default (Loader) and a named export (PageLoader). You import them differently: `import Loader from './Loader'` vs `import { PageLoader } from './Loader'`.

`client/src/components/common/EmptyState.jsx`

EmptyState Component

Reusable UI

```
export default function EmptyState({ icon: Icon, title, description, action }) {
  return (
    <div className="flex flex-col items-center justify-center py-16 text-center animate-fade-in">
      {Icon && (
        <div className="w-16 h-16 rounded-2xl bg-brand-50 dark:bg-brand-950 flex items-center justify-center mb-4">
          <Icon size={28} className="text-brand-500" />
        </div>
      )}
      <h3 className="text-lg font-semibold ...">{title}</h3>
      {description && <p className="text-sm text-gray-500 ...">{description}</p>}
      {action && <div className="mt-4">{action}</div>}
    </div>
  )
}
```

Compound slot pattern: The `action` prop accepts JSX (any button or link). This makes the component flexible — the caller decides what action to show, EmptyState just renders it. Example: `<EmptyState action={<Button>Upload</Button>} />`.

Conditional rendering: All four parts (icon, title, description, action) render only if provided. This makes one component work for many cases: empty search, no uploads, no bookmarks, etc.

`client/src/components/common/index.js` Barrel Export File

Module Pattern

```
export { default as Button }    from './Button'
export { default as Input }     from './Input'
export { default as Badge }     from './Badge'
export { default as Modal }     from './Modal'
export { default as Loader }    from './Loader'
export { default as Skeleton }  from './Skeleton'
export { default as Avatar }    from './Avatar'
export { default as TokenChip } from './TokenChip'
export { default as EmptyState } from './EmptyState'
export { default as ProtectedRoute } from './ProtectedRoute'
```

Barrel export: This file re-exports everything from the `common` folder. Without it, every import would need the full path: `import Button from '../components/common/Button'`. With the barrel, it becomes: `import { Button, Input, Badge } from '../components/common'`. One line to import multiple components.

`export { default as Button }`: Button.jsx uses `export default function Button`. To re-export a default as a named export, you use this syntax. The `as Button` gives it the name in the barrel's namespace.

`client/src/pages/Home.jsx` Home Page

Marketing / Landing

```
const stats = [
  { label: 'Active Learners', value: '2,400+' },
  { label: 'Topics Covered',  value: '380+'   },
  ...
]
```

```
const sampleContent = [
  { title: 'DBMS — Normalization...', creator: 'Aryan S.', type: 'video', cost: 5, ... },
  ...
]
```

Static data: The stats and sample content are hardcoded constants — not from the database. This is intentional for the landing page. Real usage stats would require a DB aggregate query which adds latency and cost. Static numbers load instantly and are good for marketing. The sample cards link to /browse anyway.

```
const { user } = useAuthStore()
// ...
<Link to={user ? '/browse' : '/register'}>
  {user ? 'Browse Topics' : 'Start Learning Free'}
</Link>
```

Conditional CTA based on auth state: If logged in, the button says "Browse Topics" and goes to /browse. If not logged in, "Start Learning Free" → /register. The Navbar also conditionally shows Login/Signup vs Upload/Profile buttons.

Key interview point: The hero section uses CSS `blur-3xl` blobs for visual depth — absolutely positioned gradient circles with blur that create an ambient glow effect common in modern SaaS designs. No images needed, pure CSS.

`client/src/pages/Login.jsx` Login Page Auth Flow

```
const [resetMode, setResetMode] = useState(false)

useEffect(() => {
  const hash = window.location.hash
  if (hash.includes('type=recovery')) {
    setResetMode(true)
  }
}, [])
```

Password reset detection: When a user clicks "Forgot password?", Supabase sends them a recovery email. Clicking that link redirects them back to `/login#type=recovery&access_token=...`. The useEffect on mount checks the URL hash for `type=recovery` and switches to "Set New Password" mode.

```
const handleForgotPassword = async () => {
  const { error } = await supabase.auth.resetPasswordForEmail(form.email, {
    redirectTo: `${window.location.origin}/login`,
  })
  toast.success('Password reset link sent! Check your email.')
}
```

Supabase password reset: `resetPasswordForEmail` triggers Supabase to send a magic reset link. `redirectTo` tells Supabase where to redirect after the user clicks the email link — back to our `/login` page which detects the recovery token in the hash.

```
const submit = async e => {
  const { data, error } = await supabase.auth.signInWithPassword({...})
  setUser(data.user)
  const { data: profile } = await supabase.from('profiles').select('*').eq('id', data.user.id)
  setProfile(profile)
  navigate('/')
}
```

Two-step login: First authenticate with Supabase (JWT is stored in localStorage automatically). Then fetch the profile row (username, token_balance, role, etc.) and put it in Zustand so every page can access it without re-querying.

Why fetch profile separately? Supabase's auth.user contains email and ID but not your custom columns like token_balance. Those live in the profiles table which you query separately.

`client/src/pages/Register.jsx`

Register Page — 2-Step OTP Flow

Auth Flow

```
const [step, setStep] = useState(1) // 1: form, 2: OTP entry
const [resendIn, setResendIn] = useState(0) // countdown seconds

useEffect(() => {
  if (resendIn <= 0) return
  const t = setInterval(() => setResendIn(s => s - 1), 1000)
  return () => clearInterval(t)
}, [resendIn])
```

Resend countdown: After sending the OTP, we set `resendIn = 30`. The useEffect fires whenever `resendIn` changes. It starts a 1-second interval that decrements the counter. At 0, the interval stops. The return cleanup clears the interval. This prevents the user from spamming verification emails.

```
const validatePassword = pw => {
  if (pw.length < 8) return 'Min 8 characters'
  if (!/[A-Z]/.test(pw)) return 'Add an uppercase letter'
  if (!/[a-z]/.test(pw)) return 'Add a lowercase letter'
  if (!/[0-9]/.test(pw)) return 'Add a number'
  if (!/[^A-Za-z0-9]/.test(pw)) return 'Add a special character'
  return null
}
```

Password strength validation: Uses regex to test each character class. Returns an error message string or null if valid. `/[^A-Za-z0-9]/.test(pw)` checks for at least one character that is NOT a letter or number (i.e., a special character like @, #, !, etc.).

```
// Step 2: submit the OTP + create the account
const verifyRes = await fetch('.../api/auth/verify-code', { body: JSON.stringify({ email, code
if (!verifyRes.ok) throw new Error('Invalid or expired code')

const { error } = await supabase.auth.signUp({ email, password, options: { data: { username }
const { data: loginData } = await supabase.auth.signInWithPassword({ email, password })
```

3-step registration: (1) Verify the custom OTP via our server. (2) Create the Supabase auth user (which triggers `handle_new_user()` DB trigger to create the profile with 50 tokens). (3) Auto-login — sign in immediately so the user doesn't have to login separately.

Why custom OTP + Supabase signUp? Supabase's built-in email verification sends its own email. We wanted a custom branded email (BrainBarter template via SendGrid)

so we handle verification ourselves. Supabase email confirmation can be disabled in the dashboard to allow this.

`client/src/pages/Browse.jsx` **Browse Page** Content Discovery

```
// Effect 1: Load subjects on mount
useEffect(() => {
  supabase.from('subjects').select('*').order('semester')
    .then(({ data }) => {
      setSubjects(data || [])
      if (data?.length) setSelectedSubject(data[0])  // auto-select first
      setLoadingSubjects(false)
    })
}, [])

// Effect 2: Reload content when selected subject changes
useEffect(() => {
  if (!selectedSubject) return
  setLoadingContent(true)
  supabase.from('content')
    .select('*, creator:profiles(username, is_verified)')
    .eq('is_published', true)
    .then(({ data }) => {
      const filtered = (data || []).filter(c =>
        !c.subject_name || c.subject_name === selectedSubject.name
      )
      setContent(filtered)
      setLoadingContent(false)
    })
}, [selectedSubject])
```

Two separate useEffects: The first runs once on mount (empty deps array) to load subjects. The second runs whenever `selectedSubject` changes. When the user clicks a different subject in the sidebar, `setSelectedSubject(s)` triggers the second effect to refetch content for that subject.

Joined query: `creator:profiles(username, is_verified)` is Supabase's syntax for a foreign key join. It fetches `username` and `is_verified` from the profiles table where `creator_id = profiles.id`. The result is a nested object: `c.creator.username`.

is_published filter: Only approved/published content shows in Browse. Creators see all their drafts in the Dashboard.

```
const filtered = content.filter(c => {
  const matchSearch = c.title.toLowerCase().includes(search.toLowerCase())
  const matchDiff   = difficulty === 'All' || c.difficulty === difficulty
  const matchType   = type === 'All' || c.type === type
  return matchSearch && matchDiff && matchType
})
```

Client-side filtering: Search/difficulty/type filters run in the browser on the already-fetched content array. No extra API calls on each filter change — instant filtering. This works well when there are under a few thousand content items. For scale, server-side filtering (Supabase query params) would be needed.

client/src/pages/Profile.jsx Profile Page

User Dashboard

```
useEffect(() => {
  if (!profile) return
  supabase.from('bookmarks').select('*, content(id, title, type)').eq('user_id', profile.id)
    .then(({ data }) => setBookmarks(data || []))
  supabase.from('progress').select('*, topic:topics(name, unit:units(subject:subjects(name)))')
    .eq('user_id', profile.id)
    .then(({ data }) => setProgress(data || []))
  supabase.from('content').select('id', { count: 'exact' }).eq('creator_id', profile.id)
    .then(({ count }) => setStats(s => ({ ...s, uploads: count || 0 })))
}, [profile])
```

Three parallel queries: All three fetch calls are initiated in the same useEffect without awaiting each other — they run in parallel. Supabase returns data via .then(), so they execute simultaneously. This is much faster than sequential awaits (total time = slowest query, not sum of all).

Deep nested join: The progress query joins topics → units → subjects to get the full path name for display. Supabase allows multiple levels of foreign key joins using nested object syntax.

count: 'exact': When you only need the count and not the actual rows, this option makes Supabase return the count in headers without fetching all row data.

```
const saveProfile = async () => {
  const { data, error } = await supabase.from('profiles')
    .update({ username: form.username, bio: form.bio })
    .eq('id', profile.id).select().single()
  if (error) return toast.error(error.message)
  setProfile(data)  // update Zustand store
  setEditing(false)
}
```

Profile update flow: Update only the allowed fields (username, bio). The RLS policy on profiles blocks changing token_balance or role. After success, update the Zustand store with the fresh data from the server — this updates the Navbar's displayed username immediately without page refresh.

`client/src/pages/CreatorDashboard.jsx`

Creator Dashboard

Content Management

```
const togglePublish = async (id) => {
  const content = uploads.find(c => c.id === id)
  const { error } = await supabase.from('content')
    .update({ is_published: !content.is_published })
    .eq('id', id)
  if (error) { toast.error('Failed to update'); return }
  setUploads(u => u.map(c => c.id === id ? { ...c, is_published: !c.is_published } : c))
  toast.success(content.is_published ? 'Unpublished' : 'Published!')
}
```

Optimistic update: After the DB update succeeds, we update the local state by mapping over the uploads array and toggling the one that changed. We don't re-fetch all content from the server — we just patch the specific item in the existing array. The spread `{ ...c, is_published: !c.is_published }` creates a new object (React needs new references for re-rendering).

```
const deleteContent = async (id) => {
  if (!confirm('Delete this content? This cannot be undone.')) return
  const { error } = await supabase.from('content').delete().eq('id', id)
  if (error) { toast.error('Failed to delete'); return }
  setUploads(u => u.filter(c => c.id !== id))
}
```

Confirmation guard: `confirm()` shows a native browser dialog. If the user clicks Cancel, the function returns early. Only on OK does the delete proceed. Simple but effective for destructive actions.

Filter to remove: `u.filter(c => c.id !== id)` creates a new array excluding the deleted item. More efficient than re-fetching the entire list from the server.

```
const totalViews = uploads.reduce((a, c) => a + (c.views || 0), 0)
```

reduce: Sums all view counts across the creator's uploads. `a` is the accumulator (starts at 0), `c` is each upload item. `(c.views || 0)` defaults to 0 if views is null/undefined.

`client/src/pages/AdminPanel.jsx` **Admin Panel** Administration

Withdrawal Handling — Double-Refund Protection

```
} else if (action === 'reject') {
  if (request.status !== 'pending') return toast.error('This request was already processed')
  const reason = prompt('Rejection reason:')
  if (!reason) return

  // Mark rejected FIRST, only if still pending
  const { data: updated, error } = await supabase.from('withdrawal_requests')
    .update({ status: 'rejected', admin_note: reason, processed_at: new Date().toISOString() })
    .eq('id', id)
    .eq('status', 'pending')  // ← atomic guard
    .select()

  if (error || !updated?.length) {
    return toast.error('Could not reject — it may already be processed')
  }
```

```
  // Now refund tokens
  const { error: refundError } = await supabase.rpc('refund_tokens', {
    p_user_id: request.user_id, p_amount: request.tokens_amount, p_reason: 'Withdrawal rejecte
  })
}
```

Double-refund prevention: This is a critical pattern. Two admins could theoretically click "Reject" simultaneously. Without the guard, both would refund the tokens, giving the user 2x the tokens back.

.eq('status', 'pending'): The SQL UPDATE includes `WHERE status = 'pending'`. If another admin already rejected it, the status is 'rejected' and this UPDATE matches 0 rows. The `.select()` returns the updated rows, and `!updated?.length` tells us nothing was changed — we show an error instead of refunding.

Order matters: Mark rejected FIRST, then refund. If we refunded first and the reject-mark failed, the user gets their tokens back AND their withdrawal is still "pending" — they could withdraw again. The current order is safe.

5 Admin Tabs

Tab	What it shows	Admin actions
Withdrawals	All withdrawal requests with user details, amount, method	Approve (enter txn ID) or Reject & Refund
Feedback	User feedback submissions with category	Mark Reviewed / Done
Reports	Content reports with reason and reporter	Resolve or Remove content
Verify Creators	All profiles sorted by token balance	Toggle is_verified badge
Content	All content (latest 50) with creator info	Publish or Unpublish

`client/src/pages/Leaderboard.jsx` **Leaderboard Page** Gamification

```
useEffect(() => {
  setLoading(true)
  let query = supabase.from('token_transactions').select('user_id, amount')
  const now = new Date()
  if (period === 'week') {
    query = query.gte('created_at', new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000).toISOStr
  } else if (period === 'month') {
    query = query.gte('created_at', new Date(now.getTime() - 30 * 24 * 60 * 60 * 1000).toISOSt
  }
  query.eq('type', 'earn').then(async ({ data: txns }) => {
    const userEarnings = {}
    txns.forEach(t => { userEarnings[t.user_id] = (userEarnings[t.user_id] || 0) + (t.amount |
    const userIds = Object.keys(userEarnings).sort((a, b) => userEarnings[b] - userEarnings[a]
    const { data: profiles } = await supabase.from('profiles')
      .select('id, username, avatar_url, token_balance, is_verified')
      .in('id', userIds)
    const sorted = userIds.map(uid => profiles.find(p => p.id === uid)).filter(Boolean)
    setLeaders(sorted)
  })
}, [period])
```

Period filtering: `.gte('created_at', timestamp)` filters transactions newer than N days. For 'all', no date filter — all time. The query is built dynamically by chaining conditions.

Client-side aggregation: Fetch all earn transactions, group by user_id using a plain object as a hash map, sort by total earnings, take top 10. Then fetch profile details for those 10 users.

Sort preservation: `userIds.map(uid => profiles.find(...))` maps over the SORTED userIds array, finding each profile. This preserves ranking order since Supabase's `.in()` doesn't guarantee order.

```
// Podium layout: 2nd, 1st, 3rd left-to-right (visual pyramid)
{[top3[1], top3[0], top3[2]].map((u, i) => {
  const actualRank = i === 0 ? 2 : i === 1 ? 1 : 3
  const heights = ['h-28', 'h-36', 'h-24']  // 2nd, 1st, 3rd heights
```

Podium reordering: The visual podium shows [2nd, 1st, 3rd] from left to right — not [1st, 2nd, 3rd]. This is the classic Olympic/award podium look where 1st place stands tallest in the center. The array is reordered before mapping.

client/COMPLETE_SETUP.sql

Database Setup — Full SQL Explained

PostgreSQL + pgvector

Email Verification Codes Table

```
create table if not exists email_verification_codes (
  email      text primary key,  -- one pending code per email address
  code       text not null,
  expires_at timestamptz not null,
  created_at timestamptz default now()
);
alter table email_verification_codes enable row level security;
create policy "No client access to verification codes" on email_verification_codes
  for all using (false);  -- blocks ALL client access; server uses service_role which bypasses
```

email as primary key: Each email can only have ONE pending verification code at a time. If you request a new code, the `upsert({ onConflict: 'email' })` in authController replaces the old row. This prevents stale codes from being valid.

RLS blocks all clients: `using (false)` evaluates to false for ALL rows for client connections. Only the server's service_role key bypasses RLS entirely — clients can never read or write this table directly.

Profiles Table + RLS

```
create table if not exists profiles (
  id            uuid primary key references auth.users(id) on delete cascade,
  username      text,
  email         text,
  bio           text,
  avatar_url    text,
  token_balance integer default 50,  -- everyone starts with 50 free tokens
  is_verified   boolean default false,
  role          text default 'user' check (role in ('user','admin')),
  created_at    timestamptz default now()
);

-- Users can update THEIR row, but CANNOT change token_balance or role
create policy "Users can update own profile" on profiles
  for update using (auth.uid() = id)
  with check (
```

```
    auth.uid() = id
    and token_balance = (select token_balance from profiles where id = auth.uid())
    and role          = (select role          from profiles where id = auth.uid())
  );
```

references auth.users(id) on delete cascade: Foreign key to Supabase's auth users table. When a user deletes their account, their profile is auto-deleted too. Cascade prevents orphan rows.

WITH CHECK: This is the row-level security magic. The `using` clause controls which rows can be targeted. The `with check` clause validates the NEW values being written. Even if a user targets their own row, they can't SET token_balance to something different from the current value — the subquery fetches the current balance and forces it to match. Same for role. This means the UI can call `supabase.from('profiles').update({ username: 'newname' })` safely — but `update({ token_balance: 9999 })` from the client would be rejected by RLS.

Auto-Create Profile Trigger

```
create or replace function handle_new_user()
returns trigger as $$
begin
  insert into profiles (id, email, username, token_balance)
  values (
    new.id,
    new.email,
    coalesce(new.raw_user_meta_data->>'username', split_part(new.email, '@', 1)),
    50
  )
  on conflict (id) do nothing;  -- idempotent: re-running is safe
  return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function handle_new_user();
```

AFTER INSERT trigger: Fires automatically after every new row in auth.users (which Supabase creates when signUp() is called). This auto-creates the profile without any explicit server call.

raw_user_meta_data: When calling `supabase.auth.signUp({ options: { data: { username: 'Abhay' } } })`, the data object is stored in

`raw_user_meta_data` as JSON. The trigger extracts it with `->>'username'` (JSON text extraction).

COALESCE: Returns the first non-null value. If no username in metadata, fall back to the email's local part (before @).

SECURITY DEFINER: The function runs with the privileges of the user who DEFINED it (superuser), not the user who triggers it. This allows the trigger to insert into profiles even from anonymous auth context.

ON CONFLICT DO NOTHING: If for some reason the profile already exists (race condition, duplicate trigger), the insert is silently skipped rather than throwing an error.

credit_purchase — Idempotent Token Purchase

```
create or replace function credit_purchase(
  p_user_id uuid, p_tokens integer, p_inr text, p_payment_id text
) returns integer as $$
declare v_balance integer;
begin
  -- Guard: if this payment was already processed, return current balance
  if exists (select 1 from purchases where payment_id = p_payment_id) then
    select token_balance into v_balance from profiles where id = p_user_id;
    return v_balance;
  end if;

  insert into purchases (user_id, payment_id, tokens, inr) values (p_user_id, p_payment_id, p
  update profiles set token_balance = token_balance + p_tokens where id = p_user_id returning
  insert into token_transactions (user_id, type, amount, reason) values (p_user_id, 'earn', p
  return v_balance;
end;
$$ language plpgsql security definer;
```

Idempotency: The purchases table has a UNIQUE constraint on payment_id. If the same payment_id arrives twice (network retry, double-click), the EXISTS check returns true and we return the current balance without crediting again. This prevents double-crediting even if the payment verify endpoint is called multiple times.

RETURNING: `UPDATE ... RETURNING token_balance INTO v_balance` gets the NEW balance in a single SQL statement — no second SELECT needed.

3 operations in one function: Insert into purchases, update profile balance, insert transaction log — all in one atomic function. If any step fails, Postgres rolls back all of them.

request_withdrawal — Atomic Token Deduction

```
create or replace function request_withdrawal(p_tokens integer, p_method text, p_inr numeric,
declare v_user uuid := auth.uid(); v_bal integer;
begin
  if v_user is null then raise exception 'Not authenticated'; end if;
  select token_balance into v_bal from profiles where id = v_user;
  if v_bal < p_tokens then raise exception 'Insufficient token balance'; end if;
  if p_tokens < 625 then raise exception 'Minimum withdrawal is 625 tokens'; end if;

  update profiles set token_balance = token_balance - p_tokens where id = v_user;
  insert into withdrawal_requests (user_id, tokens_amount, ...) values (v_user, p_tokens, ...)
  insert into token_transactions (user_id, type, amount, reason) values (v_user, 'spend', p_to
end;
$$ language plpgsql security definer;
```

auth.uid(): Built-in Supabase/PostgREST function that returns the authenticated user's UUID from the JWT in the request headers. This is how the function knows who is calling it even without receiving user_id as a parameter.

Balance check before deduction: Read balance → validate → deduct. Because this entire function runs in a transaction, no concurrent withdrawal can drain the balance between the check and the update. Postgres row-level locking handles this.

RAISE EXCEPTION: If balance is insufficient, raises a Postgres exception which aborts the transaction and returns an error to the client. The calling code in Wallet.jsx catches this error.

rate_content — Anti-Self-Rating

```
create or replace function rate_content(p_content_id uuid, p_stars integer) returns void as $$
declare v_user uuid := auth.uid(); v_already boolean; v_creator uuid;
begin
  select creator_id into v_creator from content where id = p_content_id;
  select exists (select 1 from ratings where user_id = v_user and content_id = p_content_id) i
  insert into ratings (user_id, content_id, stars) values (v_user, p_content_id, p_stars)
    on conflict (user_id, content_id) do update set stars = excluded.stars;
  if not v_already and v_creator <> v_user then
    update profiles set token_balance = token_balance + 2 where id = v_user;
    insert into token_transactions (user_id, type, amount, reason, ref_id) values (v_user, 'ea
  end if;
end;
$$ language plpgsql security definer;
```

Two conditions for token award: `not v_already` (first time rating this content) AND `v_creator <> v_user` (not rating your own content). Both must be true. Self-rating earns 0 tokens to prevent gaming the system.

ON CONFLICT DO UPDATE: The ratings table has UNIQUE(user_id, content_id). On re-rating, instead of failing, we UPDATE the existing rating's stars. This upsert allows changing ratings while preventing duplicates.

excluded.stars: In PostgreSQL's ON CONFLICT DO UPDATE, `excluded` refers to the row that was rejected due to conflict — i.e., the new values being inserted. So `excluded.stars` = the new star rating.

pgvector — Semantic Search Setup

```
create extension if not exists vector;

create table if not exists content_chunks (
  id          bigserial primary key,
  content_id  uuid references content(id) on delete cascade,
  chunk_index int  not null,
  chunk_text  text not null,
  embedding   vector(768),  -- 768-dimensional float array
  created_at  timestamptz default now()
);

-- Approximate-nearest-neighbour index
create index if not exists content_chunks_embedding_idx
  on content_chunks using ivfflat (embedding vector_cosine_ops) with (lists = 100);

-- Semantic search function
create or replace function match_content_chunks(
  p_content_id uuid, query_embedding vector(768), match_count int default 6
) returns table (chunk_text text, chunk_index int, similarity float)
language sql stable as $$
  select chunk_text, chunk_index,
         1 - (embedding <=> query_embedding) as similarity
  from content_chunks
  where content_id = p_content_id
  order by embedding <=> query_embedding
  limit match_count;
$$;
```

vector(768): A Postgres column that stores 768 floating-point numbers. This is the embedding — a mathematical representation of text in 768-dimensional space. Semantically similar texts have embeddings that are "close" to each other in this space.

ivfflat index: Inverted File with Flat quantization. An Approximate Nearest Neighbour (ANN) algorithm. Instead of comparing the query vector against every embedding (which would be O(n)), it divides vectors into 100 clusters (lists=100). At query time, only vectors in nearby clusters are searched — much faster with some accuracy trade-off.

<=> operator: pgvector's cosine distance operator. Returns a value from 0 (identical) to 2 (opposite). We convert to similarity with `1 - distance` so higher = more relevant.

ORDER BY embedding <=> query_embedding: Sort by distance (ascending). The most relevant chunks come first. LIMIT 6 takes the top 6 most relevant passages.

STABLE: Tells Postgres this function doesn't modify the database and returns the same result for the same inputs within a single transaction. Allows query optimization.

Interview Prep Top 30 Interview Q&As Part 2 Coverage

Q: Why does Register.jsx have a 2-step OTP flow instead of just using Supabase's built-in email verification?

We wanted a custom-branded verification email with our own template and design sent through SendGrid. Supabase's built-in email sends its own plain template. So we build a custom OTP system: generate a 6-digit code server-side, store it in our `email_verification_codes` table, send via SendGrid, then verify it before calling `supabase.auth.signUp()`. Supabase email confirmation is disabled in the dashboard to allow immediate login after signUp.

Q: How does the Navbar know whether to show "Login/Signup" or the user menu?

It reads from the Zustand `useAuthStore`: `const { user, profile } = useAuthStore()`. On login, `setUser(data.user)` is called in Login.jsx which updates the store. Since the Navbar is subscribed to this store, it re-renders automatically showing the user avatar and token balance. No prop drilling or Context is needed — Zustand's global store handles this.

Q: What is the purpose of the barrel export in `components/common/index.js`?

It's a single entry point that re-exports all components. Instead of 5 separate import lines for Button, Input, Badge, Modal, Skeleton, you write one: `import { Button, Input, Badge, Modal, Skeleton } from '../components/common'`. When you rename or move a component, you only update the barrel — all files that import from common still work. It's a module organization pattern.

Q: Why does Modal.jsx lock body scroll when open?

`document.body.style.overflow = 'hidden'` prevents the background page from scrolling while the modal is open. Without this, scrolling inside the modal can accidentally scroll the page behind it on some browsers. The useEffect cleanup restores overflow to empty string when the modal closes, and the event listener for Escape key is also cleaned up to prevent memory leaks.

Q: How does the Leaderboard calculate rankings? Why not just ORDER BY token_balance in Supabase?

The leaderboard ranks by tokens EARNED in a period (week/month/all), not by current balance. A user who earned 1000 tokens and spent 900 should still rank higher than someone with 200 tokens who never spent. Ordering by `token_balance` would rank the second person higher. So we query `token_transactions` filtered by `type = 'earn'` and sum amounts per user, then sort that.

Q: What does `SECURITY DEFINER` mean in the SQL functions?

It means the function runs with the privileges of the function's CREATOR (usually superuser), not the user who CALLS it. This is needed for `credit_purchase` and `rate_content` because they update `token_balance` in profiles — which regular users can't do (blocked by RLS). The function is allowed to do it on their behalf. It's like a "trusted intermediary" that has elevated permissions for specific controlled operations.

Q: Explain the double-refund protection in AdminPanel.jsx when rejecting a withdrawal.

Two admins might click Reject simultaneously. The protection is the SQL clause `.eq('status', 'pending')` in the UPDATE query. This adds `WHERE status = 'pending'` to the SQL. If Admin A's UPDATE runs first and changes status to 'rejected', Admin B's UPDATE finds no rows matching `status = 'pending'` and updates 0 rows. The `.select()` returns empty, so `!updated?.length` is true and Admin B sees an

error "already processed". Only the first admin's rejection goes through and triggers the refund.

Q: What's the difference between `.single()` and `.maybeSingle()` in Supabase queries?

`.single()` expects exactly one row. If 0 rows are found, it throws a PostgREST 406 error. `.maybeSingle()` returns null if no row is found, without throwing an error. For queries where the row might not exist (e.g., fetching a bookmark that might not be set), use `maybeSingle()`. For cases where the row MUST exist (e.g., fetching a profile after confirmed signup), either works but error handling differs.

Q: How does the Avatar component generate initials from a name?

`name.split(' ').map(w => w[0]).join('').slice(0, 2).toUpperCase()` — splits by space, takes first character of each word, joins them, takes max 2 characters, uppercases. "Abhay Kumar" → ["A", "K"] → "AK". Single name "Abhay" → ["A"] → "A". Empty string → "" → displays "?" as fallback.

Q: Why does Browse.jsx have two separate useEffects instead of one?

They have different dependencies and timing. The first effect (`[]` deps) loads subjects ONCE on mount. The second effect (`[selectedSubject]` deps) loads content every time the selected subject changes. If they were in one effect, changing subject would also attempt to re-load subjects (unnecessary). Separate effects = separate responsibilities.

Q: What is `createRoot` in main.jsx and why is it different from the old `ReactDOM.render`?

`createRoot` is React 18's new API that enables Concurrent Mode features: automatic batching (multiple setState calls batch into one render), startTransition (mark slow updates as non-urgent), Suspense for data fetching, and streaming server-side rendering. The old `ReactDOM.render` used legacy synchronous rendering. `createRoot` makes rendering interruptible and prioritizable.

Q: How does the Resend Countdown work in Register.jsx?

After sending the OTP, `setResendIn(30)` is called. A useEffect watches `resendIn`: when it's > 0, it starts `setInterval(() => setResendIn(s => s - 1), 1000)` — decrementing every second. When it hits 0, the effect re-runs but the `if (resendIn <=`

`0) return` guard exits immediately, stopping the interval. The cleanup function `return () => clearInterval(t)` runs before each re-execution to clear the old interval.

Q: Why does Navbar have a transparent full-screen div when the user dropdown is open?

It's a click-outside-to-close pattern. When the dropdown opens, a `<div className="fixed inset-0 z-10">` is placed behind the dropdown (z-10) but above everything else. Clicking anywhere outside the dropdown clicks this invisible overlay, triggering `setUserMenuOpen(false)`. It's simpler than attaching and managing a click listener on `document` body and checking if the click target is inside the dropdown.

Q: How does the auth route (`/api/auth/send-verification`) work without `verifyToken` middleware?

During registration, the user has no JWT yet — they haven't logged in. Requiring a token on the registration endpoint would make it impossible to register. Public routes (auth, health check) have no middleware. Protected routes (AI, payment) add `verifyToken` as a middleware argument. Express calls middleware left to right: if verifyToken rejects, the handler never runs.

Q: How does Profile.jsx update the Navbar's username display immediately without a page refresh?

After saving, `setProfile(data)` is called with the fresh server response. This updates the Zustand `authStore`. Both Profile.jsx and Navbar.jsx subscribe to the same Zustand store — they both call `useAuthStore()`. When the store changes, React re-renders all components that called the hook. So the Navbar automatically re-renders with the new username.

Q: What is the <=> operator in the SQL and how does cosine distance work?

`<=>` is pgvector's cosine distance operator. Cosine distance measures the angle between two vectors — 0 means identical direction (same meaning), 2 means opposite. For semantic search, we want the smallest distance (most similar meaning). `ORDER BY embedding <=> query_embedding` sorts by smallest distance first. We convert to similarity: `1 - distance` gives a value from -1 to 1 where 1 = perfect match.

Q: Why is IVFFlat used instead of a plain index for vector search?

Without an index, finding the nearest vector requires comparing against every row — O(n) for n chunks. With IVFFlat (lists=100), vectors are clustered into 100 groups during indexing. At query time, only vectors in nearby clusters are searched — much faster (roughly O(n/100)). The trade-off is approximate results: a tiny fraction of truly nearest vectors might be in a skipped cluster. For semantic search in education, this approximation is acceptable.

Q: How do the three CreatorDashboard queries run faster by not awaiting each other?

In Profile.jsx useEffect, three Supabase calls are made with `.then()` instead of `await` in sequence. In JavaScript, `.then()` is non-blocking — all three HTTP requests are fired simultaneously. They run in parallel and each updates state independently when done. Total time = slowest query (say 200ms) not sum of all (say 600ms). This is implicit Promise.all behavior without the explicit syntax.

Q: What does `on delete cascade` mean on the profiles table's foreign key?

When the referenced row in `auth.users` is deleted (user deletes their account), PostgreSQL automatically deletes the matching profile row. Without cascade, deleting from auth.users would fail because the profiles row references it (foreign key constraint violation). Cascade makes deletion propagate to child tables, keeping the database consistent without manual cleanup code.

Q: Why does handleLogout call both `supabase.auth.signOut()` AND `logout()` from the store?

`supabase.auth.signOut()` invalidates the session on Supabase's server and removes the access token from localStorage. `logout()` clears the Zustand store (user=null, profile=null), which triggers re-renders of all subscribed components (Navbar, ProtectedRoute, etc.) to reflect the logged-out state. Without the Supabase call, the token persists in localStorage and the user would auto-login on next page load. Without the store call, the UI would still show the user as logged in.

Q: What is StrictMode and does it cause any issues in production?

StrictMode is a development-only wrapper. In development it: (1) double-invokes render functions to detect side effects in render, (2) double-invokes useEffect setup/cleanup to detect incomplete cleanup, (3) warns about deprecated React APIs. You might notice

effects running twice during development — this is intentional and expected. In production builds, StrictMode is a no-op — it adds zero overhead and causes no double renders.

Q: How does the Skeleton component make multiple text lines of different widths?

`Array.from({ length: rows })` creates an array of N empty slots to map over. Each renders a skeleton div. The last line (`i === rows - 1`) gets `w-3/4` (75% width) while all others get `w-full`. Real text paragraphs rarely end on a full-width line — this mimics natural text proportions, making the skeleton look more realistic than all-equal-width bars.

Q: How does the loading spinner in Button.jsx work without a separate image or SVG?

It's a pure CSS trick: a small div (`w-4 h-4`) with a circular shape (`rounded-full`), a 2px border, but only the TOP border colored (`border-t-transparent` makes the others transparent). With `animate-spin` rotating it 360°, the single visible border segment appears to spin. No images, no external libraries — just a rotating circle with one colored side.

Q: What is the `...props` spread in Button and Input components?

Rest props (`...props`) collect any extra HTML attributes not explicitly destructured. Spreading them onto the native element (`<button {...props}>`) forwards them automatically. This means `<Button type="submit" onClick={fn} aria-label="Save">` works without Button needing to know about these attributes. It's how custom components can be used with full HTML attribute support.

Q: How does the password reset flow work end-to-end?

(1) User enters email and clicks "Forgot password?" in Login.jsx. (2) `supabase.auth.resetPasswordForEmail(email, { redirectTo: window.location.origin + '/login' })` sends a Supabase magic link email. (3) User clicks the link → Supabase redirects to `/login#type=recovery&access_token=...`. (4) Login.jsx mounts, checks `window.location.hash`, finds `type=recovery`, sets `resetMode=true`. (5) User enters new password. (6) `supabase.auth.updateUser({ password: newPassword })` uses the token from the URL hash to authenticate and update.

Q: Why does the admin rejection flow mark the status FIRST and THEN call the refund RPC?

Safest order for preventing double-refunds. If we refunded first and the status update failed (network error), the user's tokens are returned but the request is still "pending" — admin could trigger another refund. By marking rejected first (with `.eq('status', 'pending')` guard), even if the refund RPC fails afterward, the request is in 'rejected' state and can't be refunded again. The failure surfaces as an error toast: "Marked rejected, but refund failed — refund manually."

Q: Why are all three AI routes (`/assist, /exam, /ingest`) POST methods instead of GET?

GET requests have no body — all parameters go in the URL. AI requests need to send large data: content text, user questions, content IDs. Putting these in URL params is messy, length-limited, and gets cached/logged by proxies. POST with JSON body is the right choice for sending data that triggers server-side processing. Also, these endpoints mutate state (ai_logs, content_chunks tables) which aligns with POST semantics.

Q: How does the CreatorDashboard filter content to only show the logged-in user's uploads?

`supabase.from('content').select('*').eq('creator_id', user.id)` — the `.eq('creator_id', user.id)` adds a `WHERE creator_id = user.id` clause to the SQL. Additionally, Supabase RLS policies can be configured so even without this filter, users can only read their own content. The frontend filter is a UI concern (show only your content in dashboard); RLS is the security layer (block unauthorized access even via API).

Q: What does the `data-driven navLinks array` pattern in Navbar.jsx demonstrate about clean code?

Separation of data from presentation. The nav items are defined as data (array of objects with to, label, icon). The rendering logic is separate (`.map()` producing JSX). Adding a new nav item requires adding one object to the array — zero changes to JSX. This is the same principle behind React's component model: describe what, not how. Compare to hardcoding 4 separate Link elements where adding a 5th requires duplicating JSX.

Q: How does EmptyState.jsx accept and render a button/link through the `action` prop?

In React, JSX is just JavaScript — you can pass JSX as prop values. The caller does: `<EmptyState action={<Button onClick={...}>Upload</Button>}>`. Inside EmptyState, `{action && <div className="mt-4">{action}</div>}` renders that JSX in the correct position. This is the "slot" or "render prop" pattern — the component controls layout, the caller controls the action content. EmptyState doesn't need to know anything about buttons.

Q: Explain the complete flow when a user clicks "Rate 4 stars" on a content page.

(1) ContentPage.jsx calls `supabase.rpc('rate_content', { p_content_id: id, p_stars: 4 })`. (2) The `rate_content` Postgres function runs server-side: gets creator_id of the content, checks if user already rated this content, upserts (insert or update) the rating in the ratings table. (3) If this was the FIRST rating AND the rater is NOT the creator: updates profile token_balance +2, inserts a token_transaction record. (4) Returns to frontend. (5) ContentPage re-fetches the profile to show updated balance. (6) Toast shows "+2 tokens earned!" or just "Rated 4 stars" based on `awardsTokens`.

🎓

You now know BrainBarter inside-out

Part 1 covered the server layer (controllers, middleware, utils, stores, routing). Part 2 covered the UI layer (pages, components, SQL). Together these documents explain every file, every function, and every design decision in the project. You can now walk any interviewer through the entire codebase with confidence.

— Abhay Kumar Dwivedi | BrainBarter Project